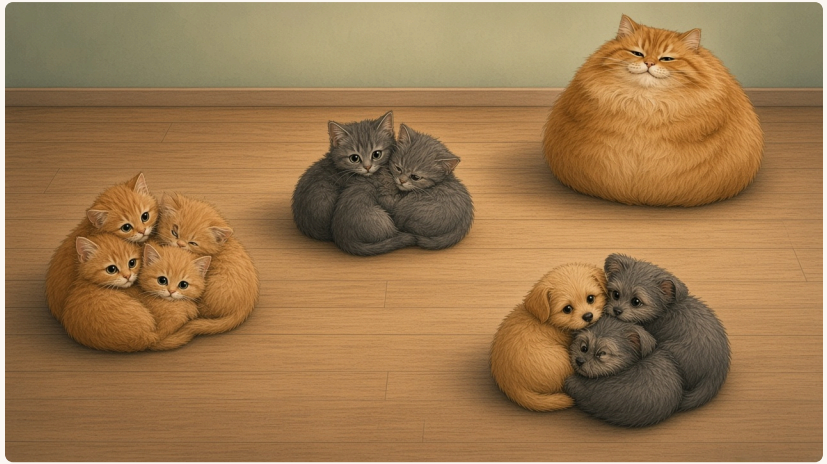
МАТЕМАТИЧЕСКИЕ МЕТОДЫ АНАЛИЗА ДАННЫХ

Кластеризация, аномалии, PCA и SVD

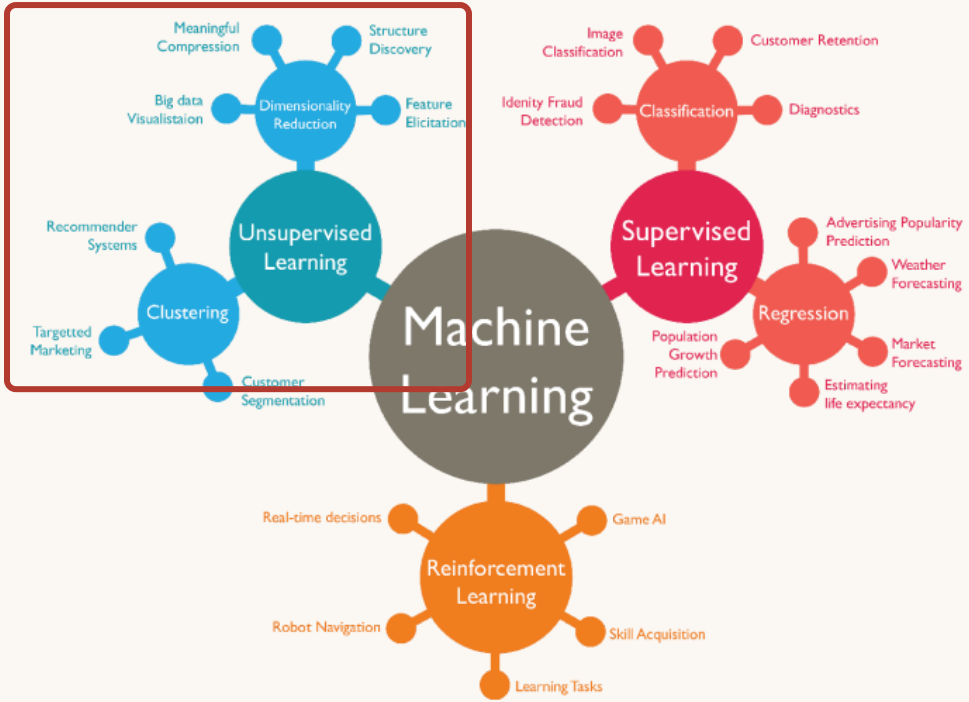
обучение без учителя: группы, сжатие и странные объекты



Лекция 8

Полина Полунина · 5 ноября 2026

Сегодня переходим в левую часть карты: обучение без учителя



Мы здесь

Лекции 2–7 — обучение с учителем (Supervised Learning, справа): у каждого объекта был правильный ответ y — цена, класс, «котик или пёсик».

Сегодня — обучение без учителя (Unsupervised Learning, слева): правильных ответов нет вовсе.

Итог: без учителя модель не может «сверяться с ответом» — она ищет закономерности в самих данных.

План: три задачи без учителя — и как выбрать метод

1. **Обучение без учителя:** что делать, если меток нет
2. **Кластеризация:** K-means, иерархическая кластеризация, DBSCAN
3. **PCA и SVD:** как описать данные меньшим числом признаков
4. **Поиск аномалий:** как найти странные объекты
5. **Итоги:** какую задачу каким методом решать

Сквозные герои




котики и пёсики потеряли метки — кластеризация



заказы доставки на карте — DBSCAN



котята и города России — PCA



кот весом 33 кг — поиск аномалий

Без учителя у нас есть только объекты X — ищем структуру в самих данных

С учителем — лекции 2–7

Есть объекты **X** и правильные ответы **y**. Модель учится предсказывать y. Качество проверяем, сравнивая прогноз с ответами.

Без учителя — сегодня

Есть только объекты **X**. Ищем структуру в самих данных. Правильных ответов нет, поэтому и качество оценить сложнее.

Три вопроса, которые можно задать данным без меток:

Есть ли в данных группы?

→ **кластеризация**: сегменты клиентов, темы текстов

Можно ли описать данные короче?

→ **PCA и SVD**: 12 признаков заменить двумя

Есть ли странные объекты?

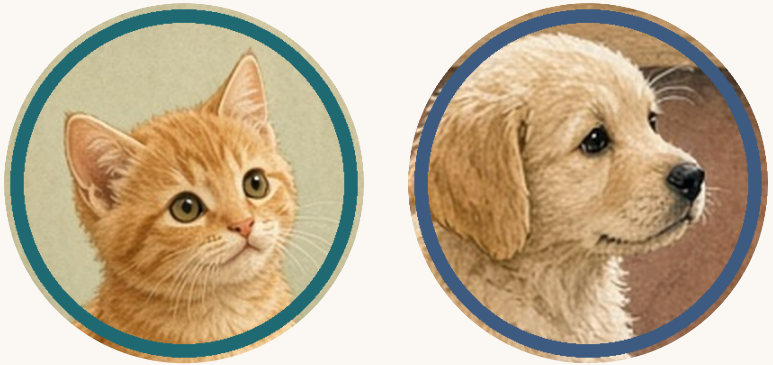
→ **поиск аномалий**: опечатки, мошенничество

> **Итог:** разметка дорогая, а неразмеченных данных много — поэтому обучение без учителя часто первый шаг анализа.

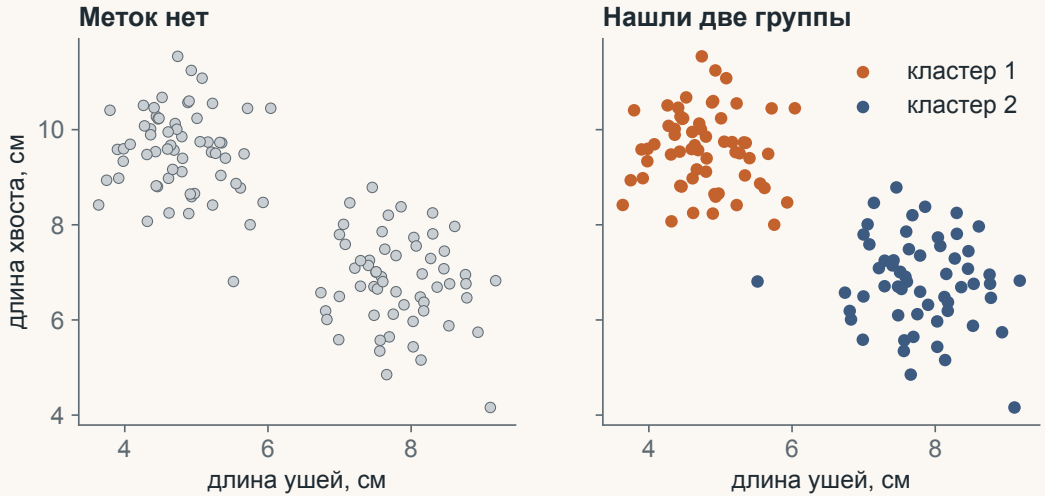
ИЩЕМ ГРУППЫ

Кластеризация

Котики и пёсики потеряли метки. Сможем ли мы сами понять, кто есть кто?



Кластеризация — разбиение объектов на группы похожих, без правильных ответов



Кластер — группа объектов, которые похожи друг на друга сильнее, чем на объекты из других групп. «Похожи» — значит **близки по расстоянию**, как соседи в kNN из лекции 4.

> **Итог:** правильного ответа нет — одни и те же точки можно честно разбить и на две группы, и на три. Поэтому методов несколько: каждый по-своему понимает, что такое «кластер».

Кластеризацию применяют, когда группы заранее никто не разметил

- **Сегментация клиентов:** разным группам — разные предложения
- **Темы обращений в поддержку:** тексты → эмбеддинги → кластеры (так устроен, например, BERTopic)
- **Разметка:** сначала кластеризуем, потом размечаем кластер целиком
- **Новости в сюжеты:** статьи об одном событии собираются вместе (Яндекс Новости, Google News)
- **Музыка:** группируем песни по звучанию (темп, громкость, тембр) — получаем стили и подборки «по настроению»
- **Сжатие картинки:** K-means на цветах пикселей — пример ниже

Оригинал



8 цветов (K = 8)



3 цвета (K = 3)



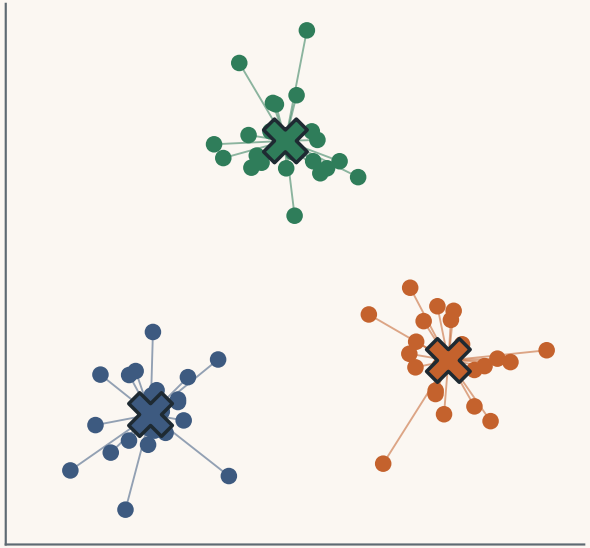
Как сжимают картинку: каждый пиксель — точка в пространстве (R, G, B). Ищем K групп похожих цветов, и каждый пиксель перекрашиваем в «средний» цвет своей группы.

K-means: каждый кластер — это центр, объект относится к ближайшему центру

Число кластеров **K** задаём сами. Ищем K центров так, чтобы объекты лежали **как можно ближе к своим центрам**:

$$J = \sum_{i=1}^{\ell} \|x_i - c(x_i)\|^2 \;\to\; \min$$

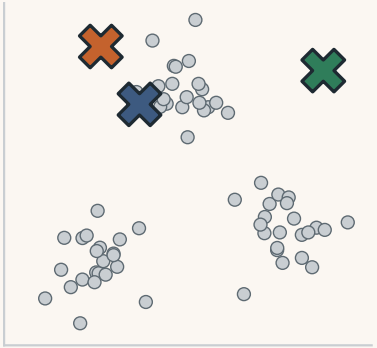
- x_i — объект, ℓ — число объектов
- $c(x_i)$ — ближайший к объекту центр
- $\|x_i - c(x_i)\|^2$ — квадрат расстояния до него (линии на рисунке)
- J — сумма квадратов: чем меньше, тем кучнее кластеры



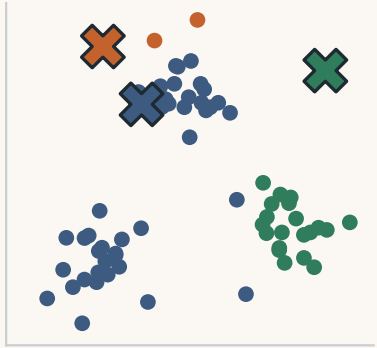
Итог: точный минимум J перебором не найти — разбиений астрономически много. J уменьшают по шагам — алгоритм на следующем слайде.

Алгоритм K-means: два шага повторяются, пока группы не перестанут меняться

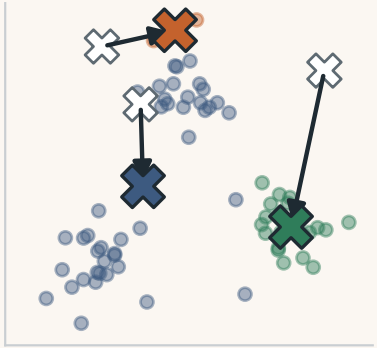
Старт:
K = 3 случайных центра



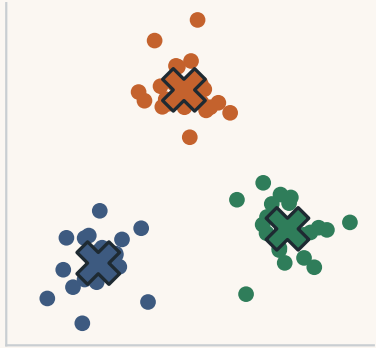
Шаг 1: каждая точка — к ближайшему центру



Шаг 2: центр — в среднее точек своей группы



Повторяем шаги 1–2, пока группы меняются



Старт Выбираем K центров — например, K случайных объектов. Здесь K = 3 (крестики).

Шаг 1 Каждую точку относим к **ближайшему** центру. Получаем K групп — они показаны цветом.

Шаг 2 Каждый центр переносим в **среднее арифметическое** точек своей группы (стрелки).

Стоп: повторяем шаги 1 и 2, пока группы меняются. Здесь хватило 4 повторений. Это **алгоритм Ллойда** — его обычно и называют K-means.

Пример руками: делим 6 котят по весу на 2 группы



Данные

Вес шести котят, кг: 0.8, 1.0, 1.4, 2.9, 3.2, 3.7. Признак один, поэтому расстояние — разница весов: от 0.8 до 1.4 оно 0.6.

Задача

K = 2. Глазами видно: три малыша и три подростка. Проверим, найдёт ли это K-means сам.

Старт

Центры — два котёнка: **0.8** и **1.4**. Нарочно неудачно: оба среди малышей. Исправится ли алгоритм?

Итерация 1: относим котят к ближайшему центру, затем пересчитываем центры

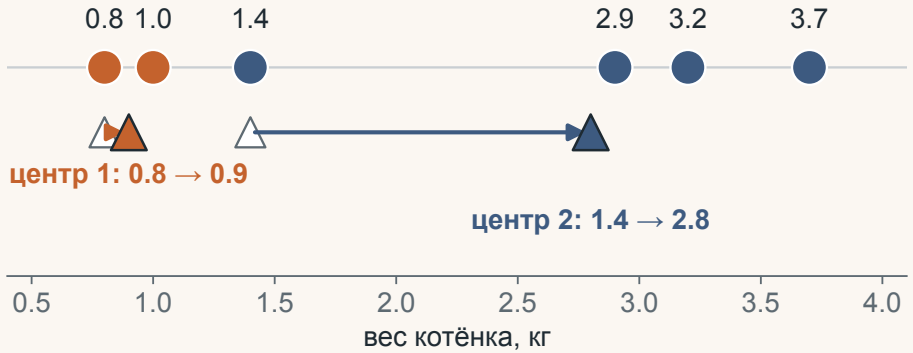
Шаг 1 Расстояния до центров. Котёнок уходит туда, где расстояние меньше (жирное):

вес	до 0.8	до 1.4	группа
0.8	**0**	0.6	1
1.0	**0.2**	0.4	1
1.4	0.6	**0**	2
2.9	2.1	**1.5**	2
3.2	2.4	**1.8**	2
3.7	2.9	**2.3**	2

Шаг 2 Новый центр — среднее своей группы:

центр 1 $= (0.8 + 1.0)/2 = 0.9$

центр 2 $= (1.4 + 2.9 + 3.2 + 3.7)/4 = 2.8$



Треугольники — центры, стрелки — куда они переехали на шаге 2.

Что произошло: центр 2 «уехал» с 1.4 на 2.8 — к подросткам. Теперь котёнку 1.4 кг ближе центр малышей.

Итерация 2: котёнок 1.4 кг переходит к малышам — и группы больше не меняются

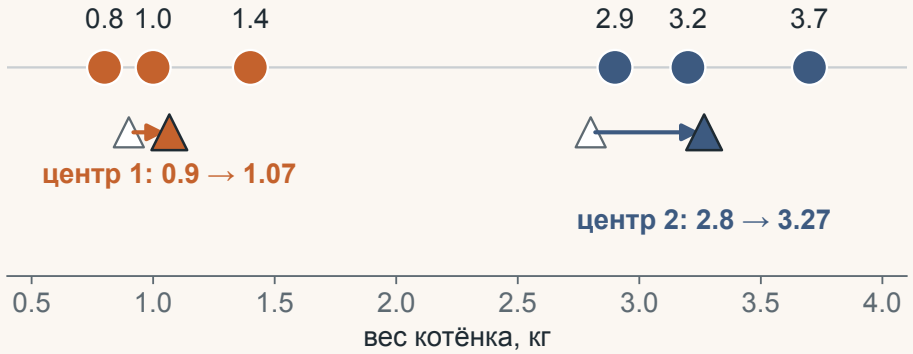
Шаг 1 Расстояния до новых центров 0.9 и 2.8:

вес	до 0.9	до 2.8	группа
0.8	**0.1**	2.0	1
1.0	**0.1**	1.8	1
1.4	**0.5**	1.4	1 ← был во 2-й
2.9	2.0	**0.1**	2
3.2	2.3	**0.4**	2
3.7	2.8	**0.9**	2

Шаг 2 Новые центры:

центр 1 $= (0.8 + 1.0 + 1.4)/3 \approx 1.07$

центр 2 $= (2.9 + 3.2 + 3.7)/3 \approx 3.27$



Итерация 3: с центрами 1.07 и 3.27 каждый котёнок остаётся в своей группе → **стоп**.

Итог: {0.8, 1.0, 1.4} и {2.9, 3.2, 3.7} — малыши и подростки, как и видно глазами. Алгоритм сам исправил плохой старт.

K-means всегда останавливается: ни один из двух шагов не увеличивает J

Шаг 1: объект — к ближайшему центру

Объект выбирает самый близкий центр, поэтому его расстояние до центра не может вырасти.

Шаг 2: центр — в среднее группы

Среднее — точка, для которой сумма квадратов расстояний до точек группы наименьшая. Поэтому и в листе регрессионного дерева (лекция 4) стоит среднее.

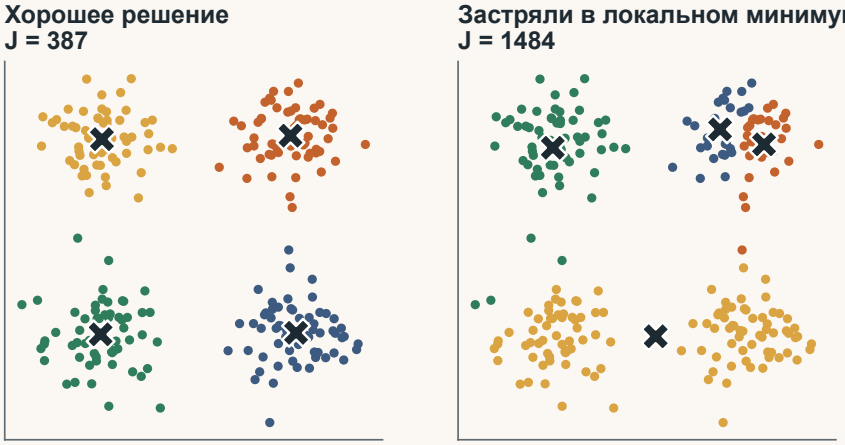
J на котятах — сумма квадратов расстояний от котят до центров их групп.
После шага 1 итерации 1: $0^2 + 0.2^2 + 0^2 + 1.5^2 + 1.8^2 + 2.3^2 = 10.82$.

после	шага 1 (итер. 1)	шага 2 (итер. 1)	шага 1 (итер. 2)	шага 2 (итер. 2)
J	**10.82**	**2.96**	**1.25**	**0.51**

Каждый раз, когда группы меняются, J строго уменьшается (при равных расстояниях объект остаётся в своей группе). Значит, разбиения не повторяются, а их конечное число — алгоритм остановится.

Но: останавливается он в **локальном** минимуме — результат зависит от стартовых центров.

Результат зависит от старта, поэтому K-means запускают несколько раз и берут лучший



Справа два центра стартовали в одном облаке: его разрезало пополам, а два других облака слиплись. Ни шаг 1, ни шаг 2 это не исправят — алгоритм остановился, хотя J почти в 4 раза больше.

Эксперимент: 200 запусков

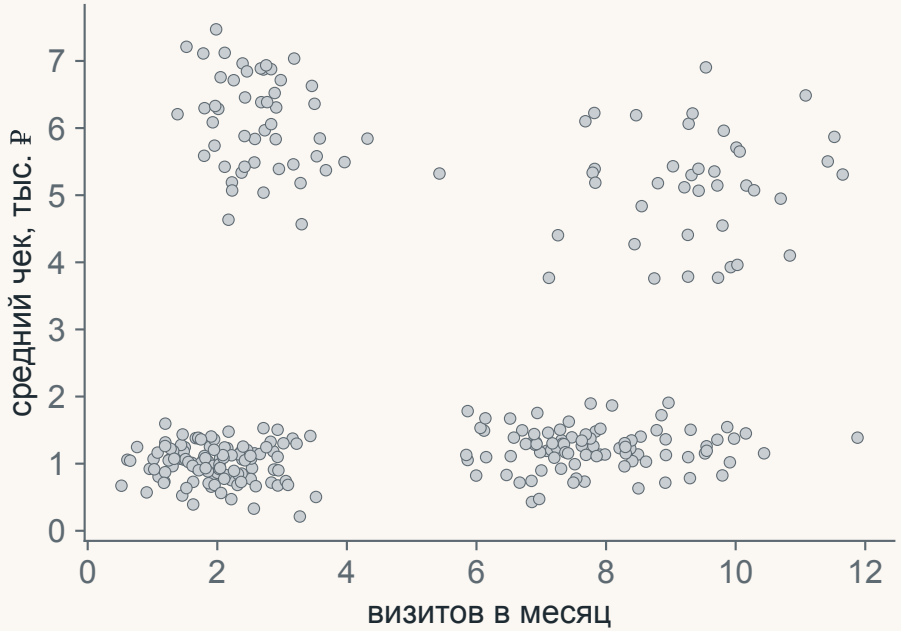
Со случайным стартом **18.5%** запусков застряли в плохом решении. Со стартом k-means++ — **1 из 200**.

Старт k-means++

Первый центр — случайный объект. Каждый следующий тоже выбираем случайно, но с вероятностью, пропорциональной **квадрату расстояния** до ближайшего из уже выбранных центров. Далёкие точки выбираются чаще — центры расходятся по разным облакам.

В sklearn: `KMeans(n_clusters=4, n_init=10)` — 10 запусков, берём решение с наименьшим J. Старт k-means++ — по умолчанию; `n_init="auto"` (с версии 1.4) делает для него один запуск.

Как выбрать число кластеров K? Пример: клиенты зоомагазина

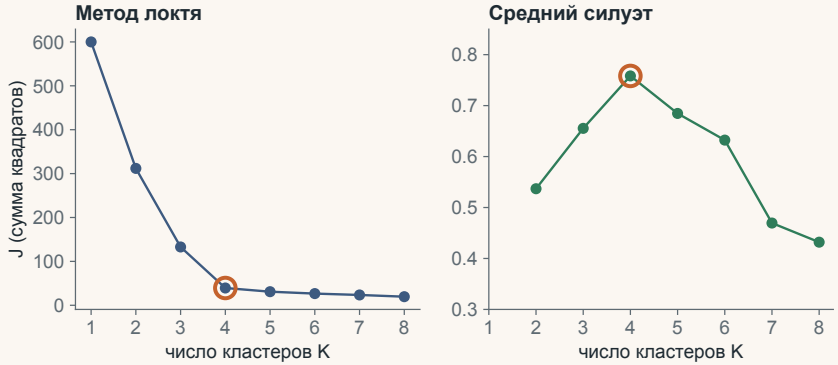


300 клиентов, два признака: сколько раз в месяц приходят и сколько в среднем тратят за визит.

Маркетингу нужны сегменты, чтобы каждому сделать своё предложение.

Вопрос залу. Сколько здесь групп? А как выбрать K, если признаков не два, а двадцать и картинку не нарисовать?

K выбирают по «локтю» графика J и по максимуму силуэта — здесь оба дают K = 4



Метод локтя. Наилучшее возможное J с ростом K только уменьшается (при K = числу объектов J = 0), поэтому просто минимизировать J нельзя. Ищем K, после которого J падает уже медленно: 600 → 312 → 133 → **39** → 31. После K = 4 выигрыш мал — это «локоть».

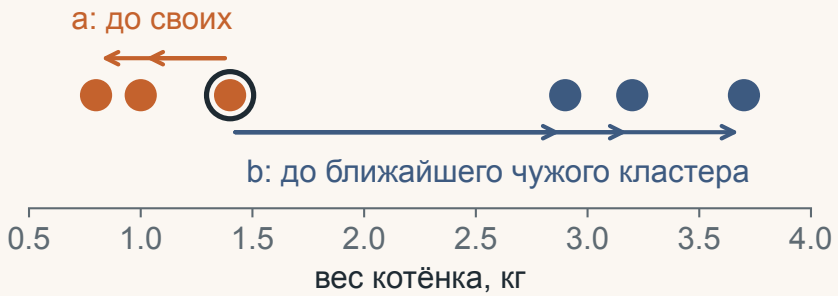
Силуэт — число от −1 до 1: насколько объекты ближе к своему кластеру, чем к соседнему (подробно — на следующем слайде). Чем больше, тем лучше. Максимум — **0.76** при K = 4.

> **Итог:** оба способа указывают на 4 сегмента. Признаки перед этим отмасштабированы — зачем, покажем на слайде «Ловушка: масштаб признаков».

Силуэт показывает, насколько объект ближе к своей группе, чем к чужой

$$s = \frac{b - a}{\max(a, b)}, \qquad -1 \le s \le 1$$

- a — среднее расстояние до **остальных** объектов своего кластера
- b — среднее расстояние до объектов **ближайшего чужого** кластера
- $s \approx 1$: объект глубоко в своём кластере; $s \approx 0$: на границе; $s < 0$: скорее всего, попал не туда



Котёнок 1.4 кг в итоговом разбиении

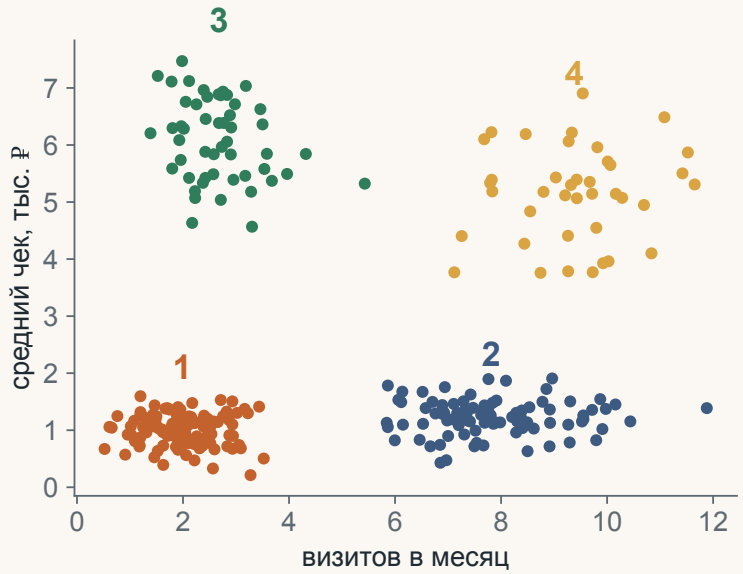
$a = (0.6 + 0.4)/2 = 0.5$ — до 0.8 и 1.0

$b = (1.5 + 1.8 + 2.3)/3 \approx 1.87$ — до 2.9, 3.2 и 3.7

$s = (1.87 - 0.5)/1.87 \approx 0.73$

Средний силуэт по всем объектам оценивает всё разбиение: {0.8, 1.0, 1.4 | 2.9, 3.2, 3.7} — **0.78**, а стартовое {0.8, 1.0 | 1.4, 2.9, 3.2, 3.7} — только **0.47**.

Кластеры нужно назвать: алгоритм выдаёт номера, смысл даёт человек

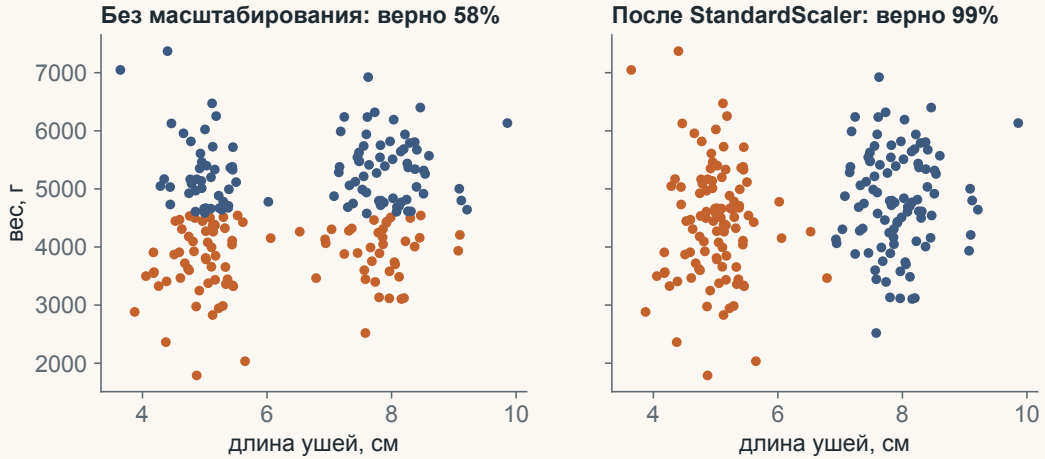


№	клиентов	визитов	чек, тыс. ₽	как назовём
1	120	2.0	1.0	заходят редко, тратят мало
2	90	7.8	1.2	постоянные: часто, за кормом
3	51	2.7	6.1	редко, но крупно: лежанки, ветеринария
4	39	9.4	5.1	лучшие клиенты: часто и много

Названия придуманы по **средним значениям признаков** в каждом кластере (столбцы «визитов» и «чек»).

Кластер — это гипотеза. Проверяем, полезна ли она: можно ли сегменту сделать своё предложение? Данные сгенерированы, и K-means восстановил настоящие сегменты почти без ошибок.

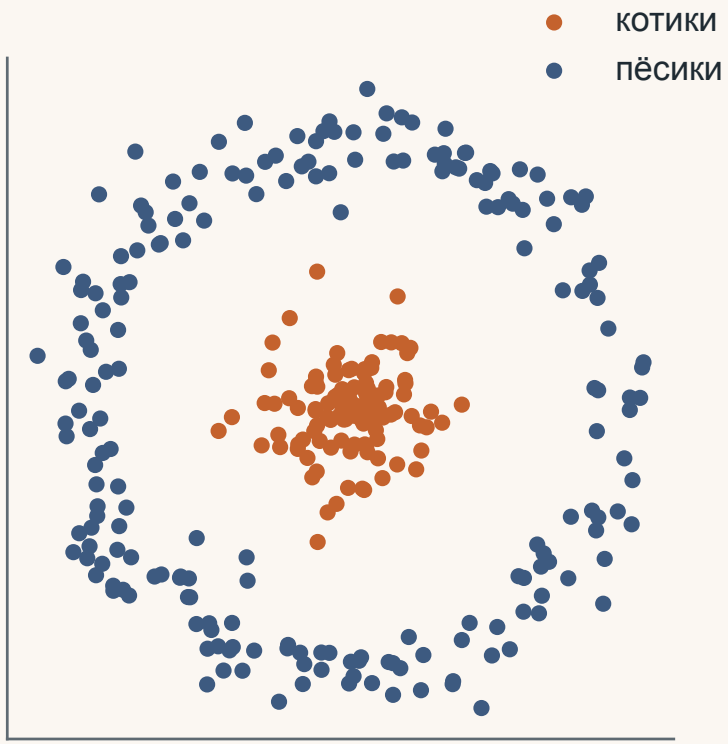
Ловушка: без масштабирования признак с большими числами решает всё



Котики и пёсики: уши в сантиметрах (разброс около 1.5 см), вес в граммах (разброс около 1000 г). Разница в весе — сотни, в ушах — единицы: расстояние почти целиком определяет вес. K-means делит животных на лёгких и тяжёлых: верно только **58%** — почти как монетка. После **StandardScaler** (у каждого признака среднее 0 и стандартное отклонение 1) — **99%**.

> **Итог:** всё, что считает расстояния — K-means, DBSCAN, kNN из лекции 4, — требует масштабирования признаков. «Верно» — доля животных, попавших в одну группу со своими.

Котики в центре, пёсики кольцом вокруг: найдёт ли их K-means?

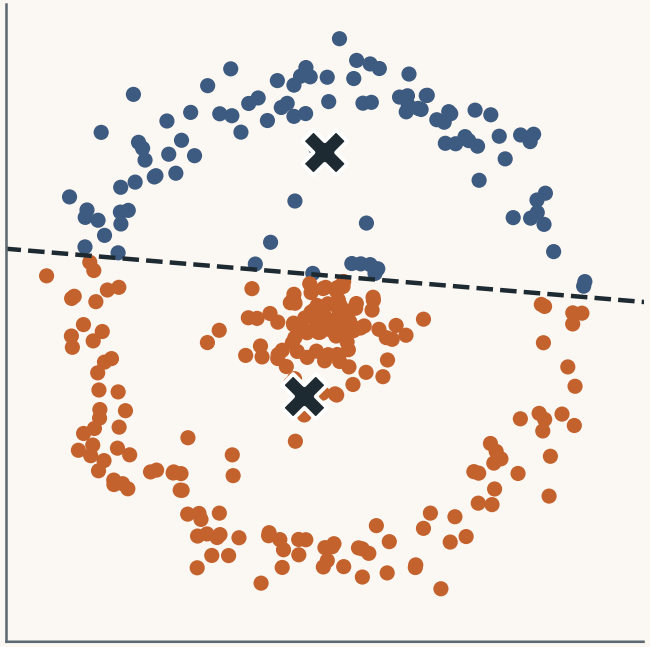


Это кольцо мы уже видели в лекциях 3 и 6 — там у каждой точки была метка «котик» или «пёсик».

Теперь меток нет: мы видим серые точки и просим K-means найти две группы.

> **Вопрос залу.** Что сделает K-means с K = 2?

K-means режет кольцо прямой линией: он умеет находить только «круглые» кластеры



Точка относится к **ближайшему** центру. Точки, одинаково далёкие от двух центров, лежат на прямой (серединный перпендикуляр, пунктир). Значит, два кластера K-means **всегда разделены прямой**. Кольцо так не отделить: верно только **62%**.

K-means хорошо находит

компактные, «круглые» кластеры похожего размера

K-means плохо находит

кольца, вытянутые и изогнутые кластеры, кластеры очень разного размера

Итог: для кластеров сложной формы нужны другие методы — иерархическая кластеризация и DBSCAN.

Иерархическая кластеризация: начинаем с отдельных объектов и шаг за шагом сливаем ближайшие

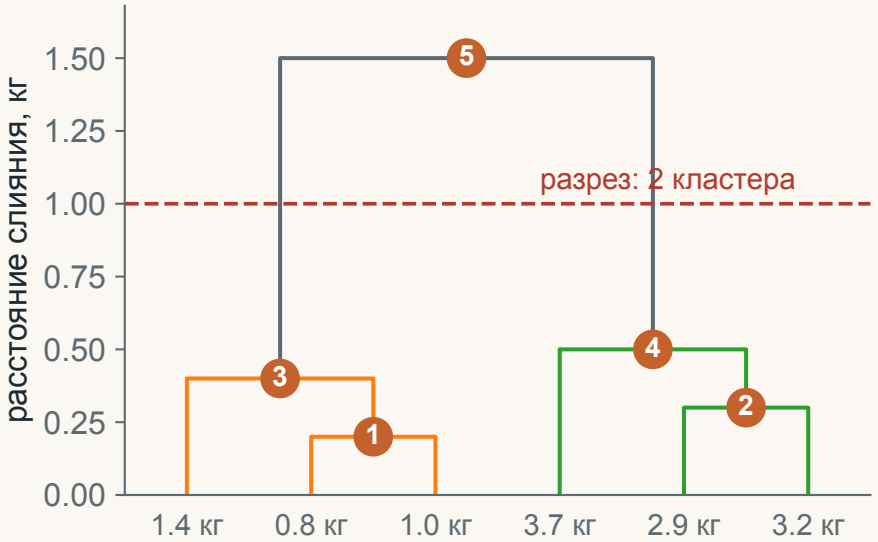
K-means требует заранее задать K и находит только «круглые» кластеры. Здесь другой подход: сначала **каждый объект — отдельный кластер**, на каждом шаге **сливаем два ближайших**, пока не останется один.

Расстояние между объектами — то же **евклидово**, что в K-means (у котят — разница весов). Новое одно: как мерить расстояние между **группами**. Здесь — по ближайшей паре объектов из двух групп; другие способы — через слайд.

шаг	что сливаем	ближайшая пара	расстояние	групп
старт	каждый котёнок — своя группа			6
1	0.8 + 1.0	0.8 и 1.0 (ближе всех)	0.2	5
2	2.9 + 3.2	2.9 и 3.2	0.3	4
3	{0.8, 1.0} + 1.4	1.0 и 1.4	0.4	3
4	{2.9, 3.2} + 3.7	3.2 и 3.7	0.5	2
5	{0.8, 1.0, 1.4} + {2.9, 3.2, 3.7}	1.4 и 2.9	1.5	1

Итог: на каждом шаге смотрим на все текущие группы и сливаем две самые близкие. Запись всех шагов — дендрограмма, на следующем слайде.

Дендрограмма — история слияний: разрежьте её на нужной высоте и получите любое K



Как читать. Внизу — котята. Каждая перекладина — одно слияние, номер в кружке — шаг из таблицы на прошлом слайде. Высота перекладины — расстояние, на котором слили две группы.

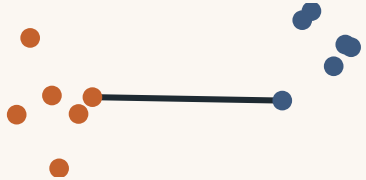
Как получить K. Проводим горизонтальный разрез: сколько вертикальных линий он пересёк, столько кластеров.

- разрез на высоте 1.0 (красный пунктир) — 2 кластера: малыши и подростки
- разрез на высоте 0.45 — 3 кластера: {0.8, 1.0, 1.4}, {2.9, 3.2} и {3.7}

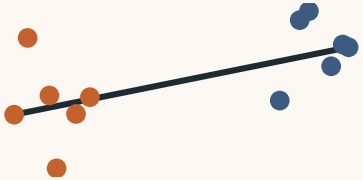
Итог: K заранее выбирать не нужно — дерево строится один раз, режем потом. Резкий скачок высоты 0.5 → 1.5 подсказывает естественное число групп: 2.

Расстояние между кластерами можно мерить по-разному — и результат будет разным

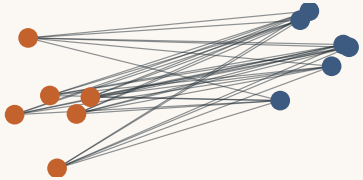
single
ближайшая пара



complete
самая далёкая пара



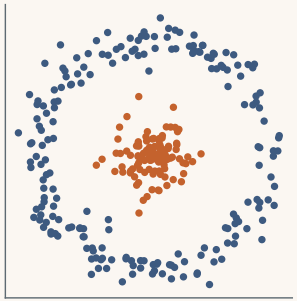
average
среднее по всем парам



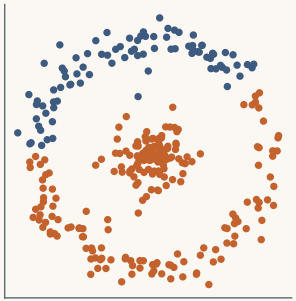
ward
рост суммы квадратов



single: верно 100%



ward: верно 64%



single (ближайшая пара) разделяет кольцо без ошибок: кластер растёт цепочкой ближайших соседей — почти так же растит кластер DBSCAN (дальше).

ward сливает группы, при которых меньше всего растёт сумма квадратов — та же J, что у K-means. Поэтому, как K-means, ищет компактные «шары» и режет кольцо.

Минусы: у single — «эффект цепочки»: пара шумных точек-мостиков склеивает разные кластеры. DBSCAN это лечит: связывает точки, только если вокруг густо. Память порядка ℓ^2 — до десятков тысяч объектов.

Идея DBSCAN: кластер — место, где точки стоят густо



Карта заказов доставки за вечер. Где здесь кластеры?

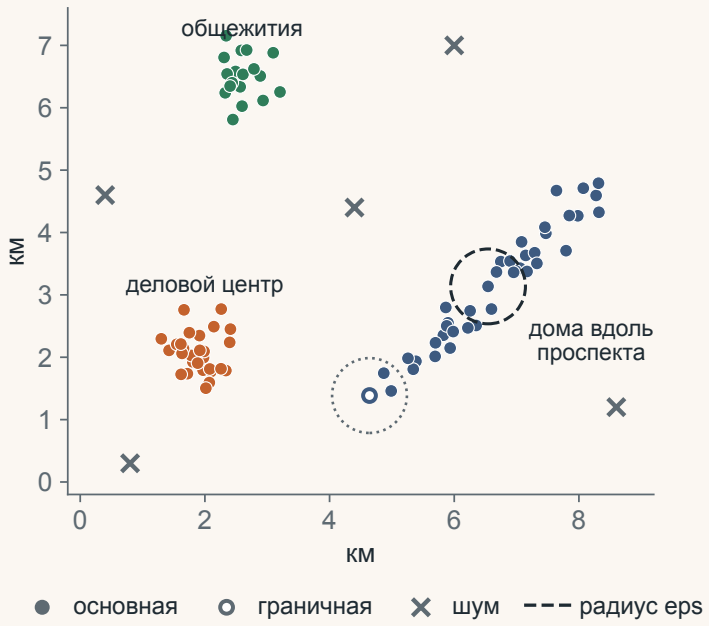
Там, где заказы идут **густо**: деловой центр, дома вдоль проспекта, общежития. Одинокий заказ на окраине не относится ни к какой группе.

Идея DBSCAN

Кластер — область, где точки стоят плотно. Кластеры разделены пустотами. Одинокие точки — шум.

DBSCAN — Density-Based Spatial Clustering of Applications with Noise, кластеризация по плотности с выделением шума. K-means здесь неудобен: число районов заранее неизвестно, проспект вытянутый, а одинокие заказы он всё равно припишет к какому-нибудь району.

DBSCAN делит точки на основные, граничные и шум — число кластеров находит сам



Два параметра: радиус **eps** и число **min_samples**.

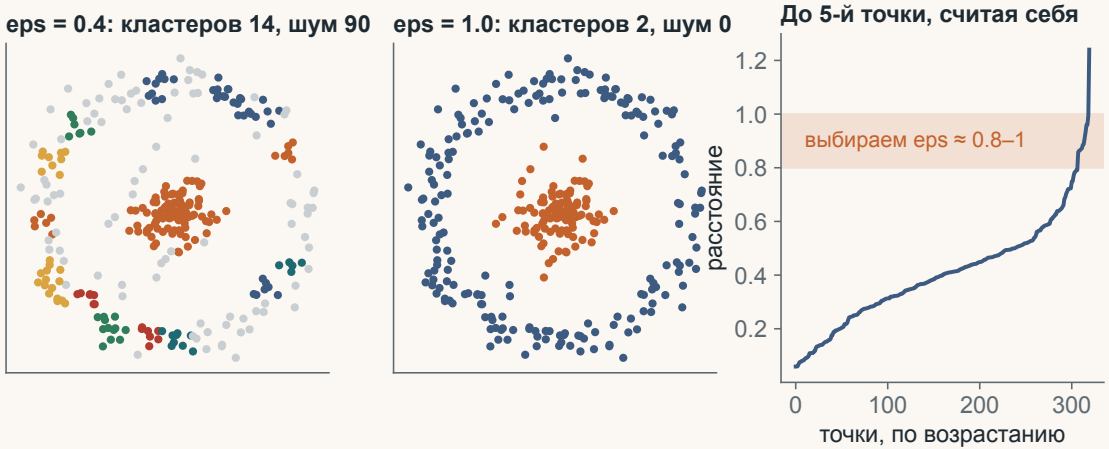
- **Основная точка:** в круге радиуса eps вокруг неё не меньше min_samples точек, считая её саму
- **Граничная:** сама не основная, но лежит в круге радиуса eps вокруг основной
- **Шум:** все остальные

Как растёт кластер:

1. берём любую основную точку без кластера;
2. добавляем всех её соседей в радиусе eps;
3. для соседей, которые сами основные, повторяем шаг 2;
4. когда добавлять некого — кластер готов, переходим к шагу 1.

Итог: заказы на карте города, eps = 0.6 км, min_samples = 4 — **3 кластера** (центр, проспект, общежития) и **5 одиноких заказов** в шуме (в sklearn метка −1). Число кластеров мы не задавали.

DBSCAN разделяет кольцо — если правильно выбрать радиус eps



eps = 1.0: кольцо и центр разделены без ошибок. **eps = 0.4:** радиус слишком мал — точки «не дотягиваются» друг до друга: 14 мелких кластеров и 90 точек шума.

Как выбрать eps. Для каждой точки считаем расстояние до 5-й ближайшей точки (считая её саму; 5 = min_samples), сортируем и рисуем (справа). Где график резко идёт вверх, начинаются одинокие точки — это и есть eps.

Итог: слишком маленький eps — почти всё шум, слишком большой — всё сливается в один кластер.

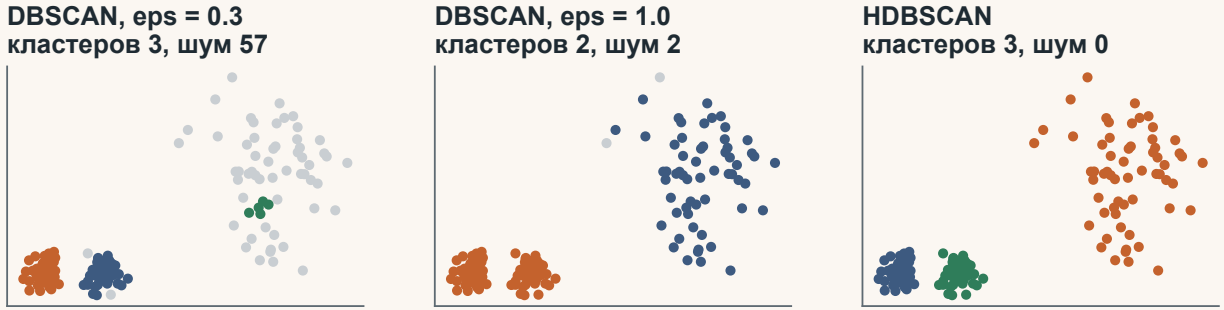
Слабое место DBSCAN — один радиус eps на все кластеры

Плюсы

- кластеры любой формы
- шум отделяется сам — пригодится в разделе про аномалии
- число кластеров задавать не нужно

Минусы

- результат сильно зависит от eps
- если кластеры разной плотности, одного eps не хватает
- как и K-means, требует масштабирования признаков



Пример: два плотных облака и одно редкое. Маленький eps отправляет редкое облако в шум (55 точек из 60; оставшиеся 5 образовали крошечный третий «кластер»), большой — склеивает плотные. **HDBSCAN** (`sklearn.cluster.HDBSCAN`) по сути перебирает eps сам и находит все три кластера.

Качество кластеризации: силуэт без меток, ARI с метками — но главное смысл

Меток нет — внутренние метрики

Силуэт: насколько объекты ближе к своим, чем к чужим.

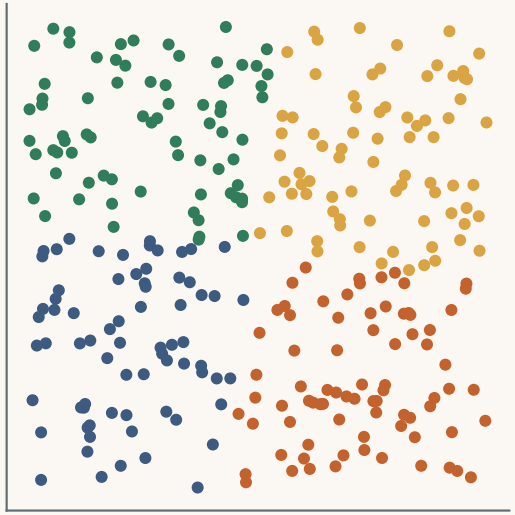
Метки есть — внешние метрики

ARI (adjusted Rand index) сравнивает два разбиения: 1 — совпадают, около 0 — как случайное. Номера кластеров не важны. Наши примеры: K-means на кольце — 0.04, DBSCAN — 1.00, клиенты — 0.995.

Главная проверка — смысл и устойчивость

Можно ли объяснить кластеры? Пригодятся ли они в задаче? Повторяются ли при перезапуске и на подвыборках?

Равномерный шум, K = 4
силуэт 0.40



Осторожно: K-means найдёт K кластеров даже в чистом шуме (справа), и силуэт будет 0.40 — не так уж мало по сравнению с 0.76 у клиентов.

K-means — для компактных групп, DBSCAN — для сложной формы и шума

	Задаём K?	Форма кластеров	Шум	Объём данных
K-means	да	компактные «шары»	каждый объект в каком-то кластере	миллионы (MiniBatchKMeans)
Иерархическая	нет, режем дендрограмму	зависит от способа связи	не выделяет	до десятков тысяч
DBSCAN	нет, но задаём eps	любая	метка −1	сотни тысяч

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score

X_s = StandardScaler().fit_transform(X)            # сначала масштабируем

km = KMeans(n_clusters=4, n_init=10, random_state=0).fit(X_s)
silhouette_score(X_s, km.labels_)                  # 0.76 на клиентах

db = DBSCAN(eps=0.3, min_samples=5).fit(X_s)       # eps — по графику расстояний до соседей
db.labels_                                         # −1 — шум
```

ОПИСЫВАЕМ КОРОЧЕ

PCA и SVD

Котята растут — и длина, и вес увеличиваются вместе. Нужны ли нам оба числа?



Снижение размерности: описать объект меньшим числом признаков, почти ничего не потеряв

Визуализация

12 признаков не нарисуешь, а 2 — можно

Скорость и память

меньше признаков — быстрее обучение и меньше хранить

Шум

слабые направления часто просто шум — их можно выбросить

Мультиколлинеарность

коррелированные признаки мешают линейной регрессии (лекция 2), а новые признаки PCA не коррелированы

Проклятие размерности

Бросим 1000 случайных точек в куб с d признаками и возьмём ещё одну точку-запрос. Во сколько раз самая далёкая от неё точка дальше самой близкой?

признаков d	2	10	100	1000
во сколько раз	136	4.5	1.39	1.11

При d = 1000 все точки почти одинаково далеко от запроса.

Итог: в большой размерности «ближайший сосед» теряет смысл — страдают kNN и кластеризация. Это ещё одна причина сокращать число признаков.

Длина и вес котят растут вместе — по сути это одно свойство, «размер»



40 котят, два признака: длина тела и вес. Корреляция **r ≈ 0.92** — точки лежат почти на прямой.

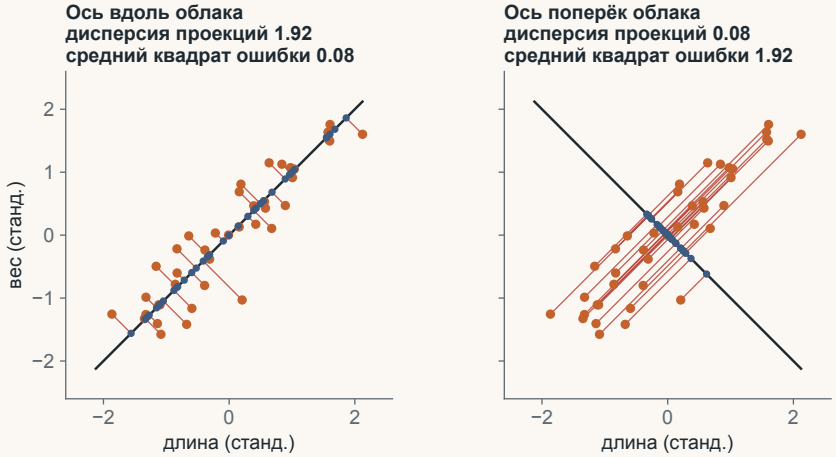
По сути оба признака измеряют одно: **насколько котёнок большой**.

Идея PCA (метод главных компонент)

Повернуть оси так, чтобы первая шла вдоль облака. Тогда почти вся информация окажется в одной новой координате — «размере котёнка».

Важно: единицы разные (см и кг), поэтому дальше признаки стандартизованы — у каждого среднее 0 и стандартное отклонение 1.

Ось вдоль облака сохраняет разброс точек и теряет мало информации



Проекция — «тень» точки на ось (синие точки). Если оставить от котёнка одно число, это будет его координата на оси.

Ошибка — расстояние от точки до оси (красные отрезки): столько мы теряем, восстанавливая точку по проекции. На рисунке — средний квадрат этих расстояний.

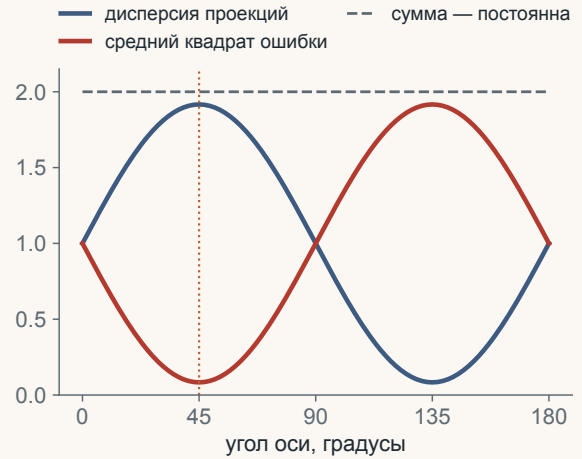
> **Сравним:** вдоль облака дисперсия проекций 1.92, а средний квадрат ошибки 0.08; поперёк — наоборот. В сумме всегда 2: дисперсия длины (1) + дисперсия веса (1).

Максимум разброса проекций = минимум ошибки: это одна и та же задача

Данные центрированы (среднее каждого признака 0), w — направление оси длины 1. По теореме Пифагора для каждой точки:

$$\|x_i\|^2 = \underbrace{\langle x_i, w\rangle^2}_{\text{проекция}^2} + \underbrace{\|x_i - \langle x_i, w\rangle w\|^2}_{\text{ошибка}^2}$$

Длина $\|x_i\|$ от выбора оси не зависит. Значит, сумма «проекция² + ошибка²» по всем точкам постоянна: что прибавилось в одном слагаемом, убыло в другом.



Крутим ось вокруг центра котят: синяя и красная кривые в сумме дают 2 при любом угле. Оптимум — 45°, ось вдоль облака.

Итог: максимизировать разброс проекций ⇔ минимизировать ошибку восстановления. PCA ищет именно такую ось.

Главные компоненты — собственные векторы матрицы ковариаций

X — центрированная матрица $\ell \times d$, $C = \frac{1}{\ell} X^T X$ — матрица ковариаций. Ищем ось w с наибольшим разбросом проекций:

$$\frac{1}{\ell} \sum_i \langle x_i, w \rangle^2 = w^T C\, w \;\to\; \max_{\|w\|=1}$$

Ответ: лучшая ось w_1

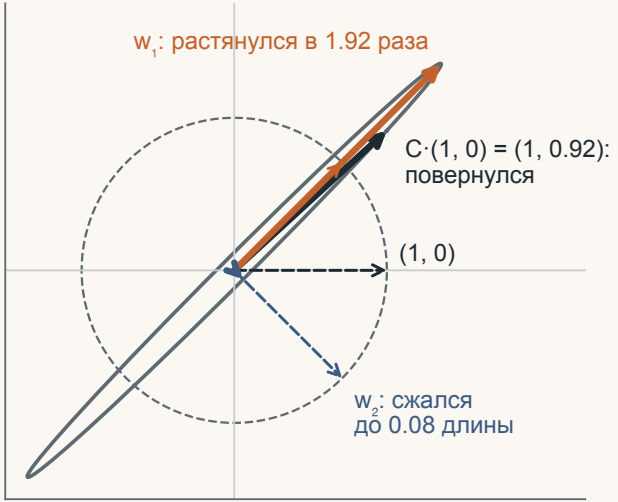
собственный вектор C с наибольшим собственным значением λ_1, а λ_1 — разброс вдоль неё. Следующие оси перпендикулярны предыдущим (вывод — в приложении).

Что в формулах

- C — матрица $d \times d$: дисперсии признаков на диагонали, ковариации вне её. Ковариация — среднее произведение отклонений двух признаков от их средних: больше 0, если признаки растут вместе
- **собственный вектор**: $Cw = \lambda w$ — матрица его только растягивает в λ раз, не поворачивая (наглядно — на следующем слайде)
- w_j — **главные компоненты**, новые оси; новые признаки — $z_{ij} = \langle x_i, w_j \rangle$

Итог: PCA — собственные векторы матрицы ковариаций с наибольшими λ. Делить на ℓ или $\ell - 1$ — на оси не влияет.

Собственный вектор матрица только растягивает: у котят это оси «размер» и «стройность»



пунктир — до умножения на C, сплошные — после

Матрица ковариаций котят C переводит круг в эллипс, обычный вектор при этом поворачивается. **Собственный вектор** — направление, которое не поворачивается: $Cw = \lambda w$. **Собственное значение** (число) λ — во сколько раз он растянулся.

- λ **— разброс:** $w^T C w = \lambda$ при $\|w\| = 1$.
- **Как найти:** $(C - \lambda I)\, w = 0$ имеет ненулевое решение, только если $\det(C - \lambda I) = 0$ (I — единичная матрица).
- **Оси перпендикулярны:** у симметричной матрицы, как C, собственные векторы можно выбрать перпендикулярными.

Итог: собственные векторы C — направления, которые она не поворачивает (оси эллипса), собственные значения — разброс вдоль них.

На котятах первая компонента «размер» сохраняет 96% разброса

После стандартизации матрица ковариаций — это матрица корреляций:

$$C = \begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix}, \quad r = 0.92$$

Собственные значения — корни уравнения $\det(C - \lambda I) = (1-\lambda)^2 - r^2 = 0$, то есть $\lambda = 1 \pm r$:

$$\lambda_1 = 1.92, \quad \lambda_2 = 0.08$$

$w_1 = (1, 1)/\sqrt{2}$ — **«размер»**

длина и вес растут вместе

$w_2 = (1, -1)/\sqrt{2}$ — **«стройность»**

длиннее, чем положено при таком весе, или наоборот

Доля первой компоненты

$\lambda_1/(\lambda_1 + \lambda_2) = 1.92/2 = 96\%$. Одно число вместо двух — теряем 4% разброса.

Нюанс: знак компоненты произволен — sklearn может выдать $(-1, -1)/\sqrt{2}$. Это та же ось, направленная в другую сторону.

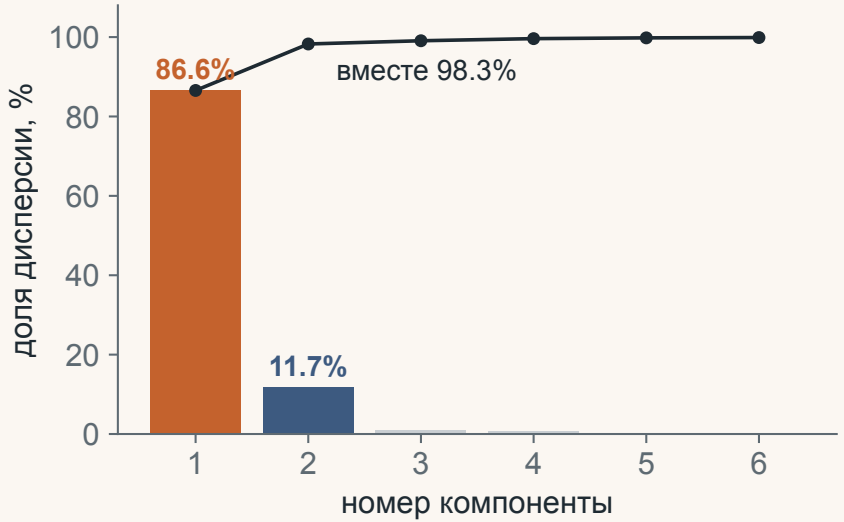
Климат 34 городов России: 12 температур, но двух компонент хватает на 98%

Каждый город — 12 чисел: средняя температура каждого месяца. Это точка в 12-мерном пространстве.

Сколько из 12 чисел действительно нужно?

Как выбрать число компонент

- набрать 90–95% дисперсии (разброса)
- найти «локоть» на графике долей
- если PCA — шаг перед моделью, по качеству модели на валидации



Первая компонента объясняет 86.6% дисперсии, вторая — 11.7%. **Вместе — 98.3%.**

Данные: NASA POWER, реанализ MERRA-2, средние за 2001–2025 гг. Это оценки по сетке около 50 км: отдельные значения могут отличаться от станционных норм, на общую картину это не влияет.

Две компоненты рисуют карту климата — хотя географии в данных нет

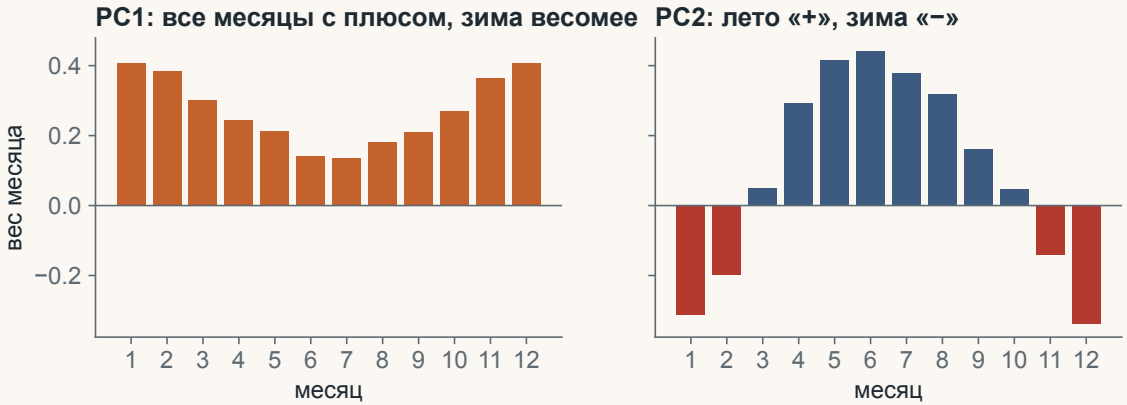


Каждая точка — город в новых координатах: PC1 и PC2 (первая и вторая главные компоненты).

В данных только 12 температур — ни широты, ни долготы. Но север и юг, Сибирь и побережья разошлись сами.

Вопрос залу. Почему Петропавловск-Камчатский и Якутск оказались на разных краях по вертикали?

PC1 — «насколько тепло», PC2 — «насколько жаркое лето для такой зимы»



PC1: все месяцы входят с плюсом, зимние — с большим весом: зимой города различаются сильнее всего. Корреляция PC1 со среднегодовой температурой — 0.99.

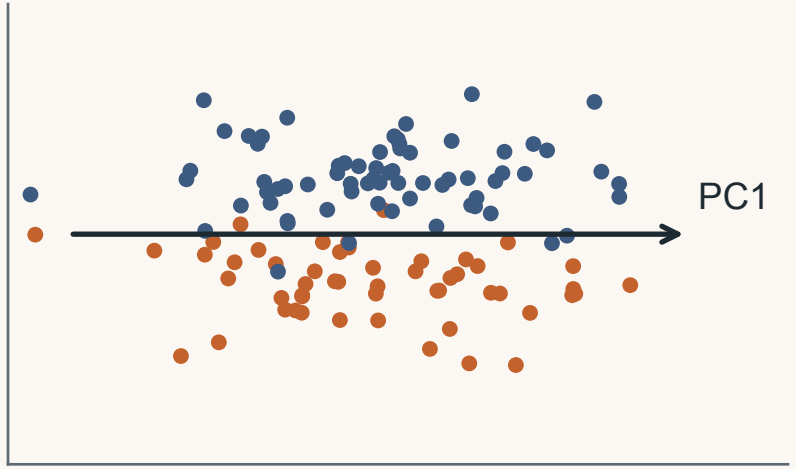
PC2: летние месяцы с плюсом, зимние с минусом. Вверху — жаркое для своей зимы лето (Якутск, Хабаровск, Астрахань), внизу — прохладное: Петропавловск-Камчатский, Мурманск, Магадан, Анадырь.

> **Как читать компоненту:** по весам признаков (`components_`) — столбики на графиках. Все признаки здесь в °C, поэтому масштабировать не стали; при разных единицах — обязательно.

Четыре ловушки PCA — и главная: PCA не смотрит на y

1. **Центрирование** обязательно — sklearn делает его сам.
2. **Разные единицы** → **StandardScaler**, как у котят. Иначе первая компонента уйдёт в признак с самыми большими числами.
3. **Обучаем только на train:** `fit` на обучении, `transform` на тесте. Иначе утечка, как со стекингом в лекции 6.
4. **PCA линейный:** поворачивает оси, но не «распрямляет» изогнутые структуры.

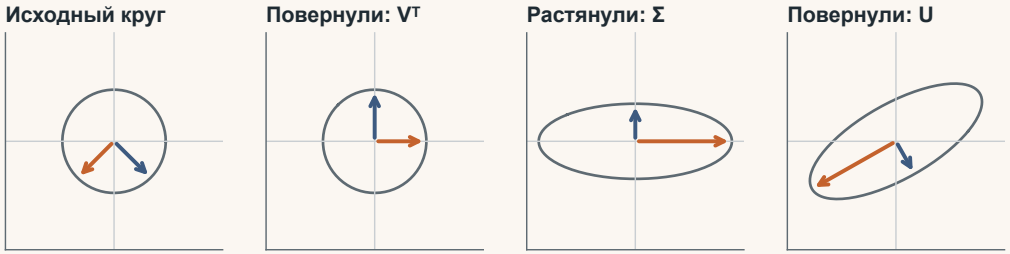
PCA не смотрит на y. Здесь классы различаются по «тонкому» направлению. PCA оставит длинное направление — и выбросит всё полезное.



Итог: если PCA — шаг перед моделью, число компонент подбираем по качеству модели, а не только по доле дисперсии.

SVD: любую матрицу можно разложить на поворот, растяжение и ещё поворот

$$X = U\,\Sigma\,V^T \qquad (\ell \times d) = (\ell \times r)(r \times r)(r \times d)$$



Сингулярные числа σ_j

$\Sigma = \mathrm{diag}(\sigma_1 \geq \cdots \geq \sigma_r > 0)$: растяжения вдоль осей эллипса, вне диагонали нули.

Ранг r

Число линейно независимых столбцов = число ненулевых σ_j. «Вес в г» рядом с «весом в кг» ранг не добавит.

Ортонормированные столбцы

Длины 1 и попарно перпендикулярны: $U^T U = I$. Длины и углы не меняются — поворот (или отражение).

Итог: любая матрица — это поворот V^T, растяжение Σ и поворот U. Сингулярное разложение (SVD) есть у **любой** матрицы — не нужны ни квадратность, ни симметричность.

PCA — это SVD центрированной матрицы данных

Пусть X центрирована и $X = U\Sigma V^T$. Так как $U^T U = I$:

$$X^T X = V\Sigma U^T U \Sigma V^T = V\,\Sigma^2\,V^T$$

Значит, $X^T X\, v_j = \sigma_j^2\, v_j$ — столбцы V собственные векторы матрицы ковариаций:

- столбцы V — **главные компоненты**
- разброс вдоль компоненты: $\lambda_j = \sigma_j^2/\ell$
- новые координаты объектов: $Z = XV = U\Sigma$

На практике: SVD самой X, без $X^T X$

Число обусловленности $\kappa = \sigma_1/\sigma_r$ — во сколько раз самое сильное растяжение больше самого слабого; примерно во столько раз усиливаются ошибки округления. У $X^T X$ сингулярные числа σ_j^2, значит, и κ в квадрате: 10^8 превращается в 10^{16} — это предел точности float64, маленькие компоненты тонут в погрешностях.

`sklearn.decomposition.PCA` выбирает сам: если объектов в 10+ раз больше признаков, с версии 1.5 считает через матрицу ковариаций — быстрее.

Итог: PCA и SVD — одна математика. Через SVD PCA считают устойчивее.

Оставим k самых больших сингулярных чисел — получим лучшее приближение ранга k

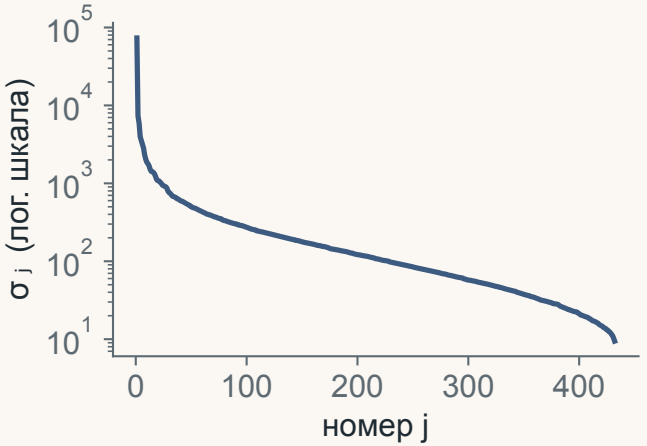
Выбросим маленькие сингулярные числа и оставим только $k < r$ самых больших:

$$X \approx X_k = U_k \, \Sigma_k \, V_k^T$$

Теорема Эккарта–Янга

Среди всех матриц ранга не выше k матрица X_k ближе всего к X — по сумме квадратов разностей элементов.

Память: храним k столбцов U, k чисел σ и k строк V^T — всего $k(\ell + d + 1)$ чисел вместо $\ell \cdot d$.



Сингулярные числа фотографии кота со следующего слайда: $\sigma_1 \approx 73\,000$, а $\sigma_{50} \approx 500$. Почти вся «энергия» — в первых числах.

Итог: если сингулярные числа быстро убывают, матрицу можно хранить гораздо компактнее почти без потерь.

Ранг 20 вместо 432: 8% памяти — и кот всё ещё узнаваем

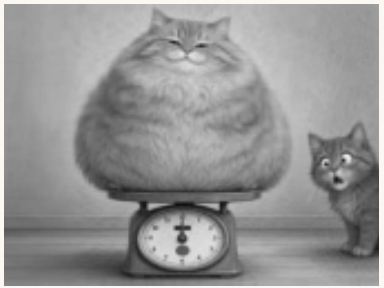
ранг 5: 2% памяти



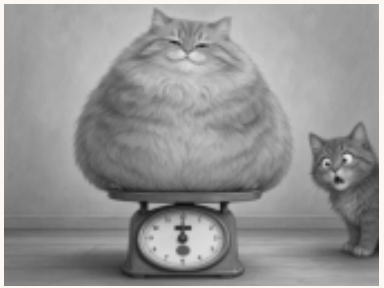
ранг 20: 8% памяти



ранг 50: 20% памяти



оригинал (ранг 432)



Чёрно-белая картинка 432 × 576 — это матрица яркостей пикселей. Оставляем k сингулярных чисел: ранг 5 — 2% памяти, ранг 20 — 8%, ранг 50 — 20%. Относительная ошибка восстановления: 11.8%, 7.0% и 4.5%.

> **Для сравнения:** JPEG устроен иначе — косинусное преобразование блоков 8 × 8, — но идея та же: хранить главное и выбрасывать мелочи.

Рекомендации: заполняем пропуски в таблице «котик × корм» приближением ранга 2

	лосось	тунец	говядина	курица	кролик
Барсик	5	4	**2.0**	2	2
Мурка	5	4	2	3	**2.7**
Пушок	**1.9**	2	5	**4.3**	5
Рыжик	3	**3.3**	3	3	3
Снежок	5	5	2	**3.1**	3
Тиша	2	**2.5**	5	4	4

Чёрным — оценки от 1 до 5, **оранжевым** — прогноз для кормов, которые котик не пробовал.

Каждому котику — 2 числа «вкусов» p_i, каждому корму — 2 числа «свойств» q_j. Оценка ≈ среднее μ плюс их скалярное произведение:

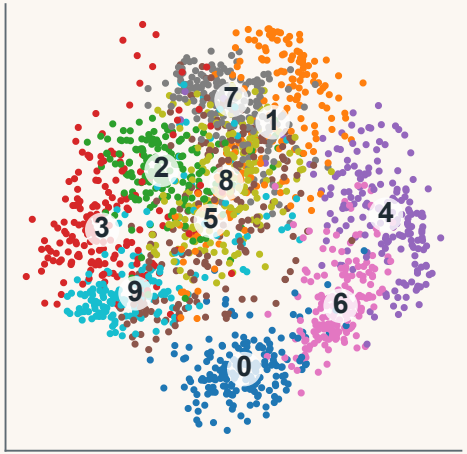
$$\text{оценка}_{ij} \approx \mu + \langle p_i, q_j \rangle, \quad p_i, q_j \in \mathbb{R}^2$$

Факторы выучиваются сами и часто читаются: «любит рыбное», «любит мясное». Определены они с точностью до поворота.

Как учат: обычное SVD не умеет работать с пропусками, поэтому подбирают p_i, q_j только по известным клеткам (ALS или градиентный спуск) с регуляризацией. Так устроены классические рекомендательные системы; метод прославился на Netflix Prize.

t-SNE и UMAP хороши для картинки, но расстояниям на ней верить нельзя

PCA: 2 компоненты



t-SNE



Рукописные цифры, 64 признака (пиксели 8 × 8). PCA — линейный метод: классы на картинке перекрываются.

t-SNE и **UMAP** нелинейные: стараются сохранить, кто чей сосед, — и классы разошлись.

Осторожно: расстояния между кластерами и их размеры на такой картинке почти ничего не значат. Картинка меняется от параметров (perplexity) и random_state.

Итог: t-SNE — для взгляда на данные, а не для признаков модели: у него нет `transform` для новых объектов. UMAP — в библиотеке umap-learn.

PCA и SVD в sklearn: учим только на train, число компонент — по доле дисперсии

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA, TruncatedSVD

pca = make_pipeline(StandardScaler(), PCA(n_components=0.95))  # оставить 95% дисперсии
Z_train = pca.fit_transform(X_train)                           # учим только на train
Z_test = pca.transform(X_test)

p = pca[-1]
p.explained_variance_ratio_                                    # доли дисперсии компонент
p.components_                                                  # веса признаков в компонентах

TruncatedSVD(n_components=100).fit_transform(X_sparse)         # разреженные матрицы, без центрирования
```

PCA

обычно плотные данные: признаки центрируются

TruncatedSVD

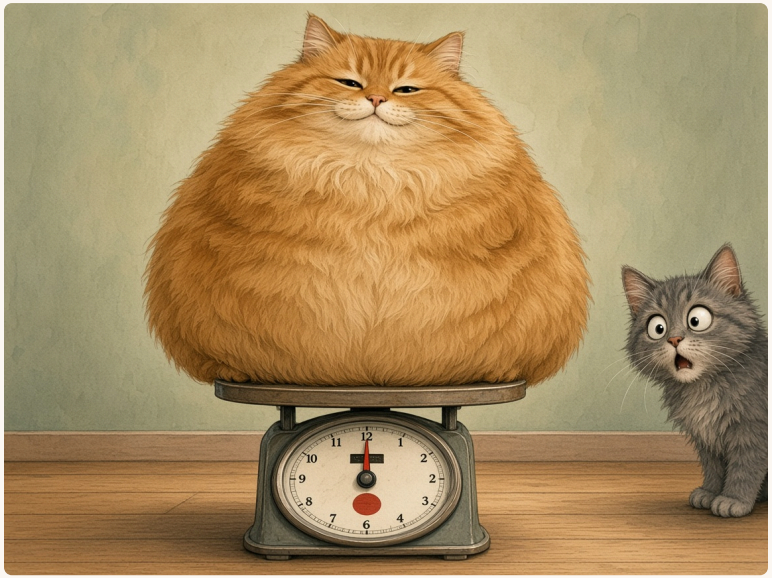
разреженные (TF-IDF текстов): центрирование сделало бы нули ненулевыми

ИЩЕМ СТРАННОЕ

Поиск аномалий

В лекции 6 линейная регрессия предсказала коту 33 кг — это была ошибка модели.

А что, если такой кот окажется прямо в данных? Например, кто-то набрал 33 вместо 3.3.



Аномалия — объект, не похожий на остальные; искать их нужно и в данных, и в потоке новых объектов

Фрод

подозрительные платежи и аккаунты

Мониторинг

всплеск ошибок, задержек или нагрузки на сервис

Ошибки в данных

опечатки и сломанные датчики — найти до обучения модели

Безопасность

вторжения в сеть, брак на производстве

Поиск выбросов (outlier detection)

Выбросы есть прямо в обучающих данных — ищем их там. Пример: опечатки в таблице весов котов.

Поиск новизны (novelty detection)

Учимся на чистых данных и проверяем каждый новый объект: похож ли он на то, что мы видели. Пример: новый платёж.

Итог: обычно меток «аномалия / норма» нет, поэтому это задача без учителя — модель описывает, как выглядит норма.

Аномалии бывают трёх типов: глобальные, контекстные и коллективные



Глобальная — странная сама по себе: кот весом 33 кг.

Контекстная — странная только в своём контексте: +5 °C в Якутске — обычный май, но в январе это аномалия.

Коллективная — каждая точка обычная, странен кусок целиком: датчик миски 30 часов показывает одно и то же — скорее всего, завис.

Итог: дальше — о глобальных аномалиях. Для остальных добавляют признаки контекста (месяц, час) или смотрят на окна ряда. Данные иллюстративные.

Если размеченных аномалий почти нет или новые не похожи на старые — нужен поиск аномалий, а не классификатор

Поиск аномалий, когда…	Классификация, когда…
размеченных аномалий очень мало (единицы) или нет совсем	примеров обоих классов достаточно
аномалии бывают разными, и новые не похожи на прошлые	новые «плохие» объекты похожи на прошлые
пример: новые схемы мошенничества, неизвестные поломки	пример: спам, известные виды брака

Классификатор учится отличать **известные** аномалии от нормы и пропустит аномалию нового вида. Поиск аномалий описывает норму и ловит всё, что на неё не похоже.

> **Совет:** даже несколько размеченных аномалий полезны — по ним выбирают порог и сравнивают методы. А когда примеров накопится много, можно перейти к классификации.

Три сигмы ломаются от самих выбросов, а квартили — нет

Правило трёх сигм

аномалия, если $z = |x - \bar{x}|/\sigma > 3$ — дальше трёх σ от среднего

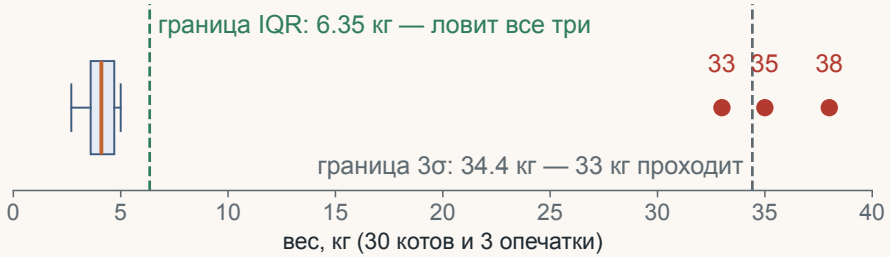
Правило IQR — «ящик с усами»

Квартили: четверть значений меньше Q_1, четверть больше Q_3; $\mathrm{IQR} = Q_3 - Q_1$. Аномалия — дальше $1.5\,\mathrm{IQR}$ за Q_1 или Q_3.

Одна опечатка. 30 котов от 2.7 до 5.0 кг и «кот» 33 кг. Среднее 4.9, σ = 5.3, $z = (33 - 4.9)/5.3 \approx 5.3 > 3$ — поймали.

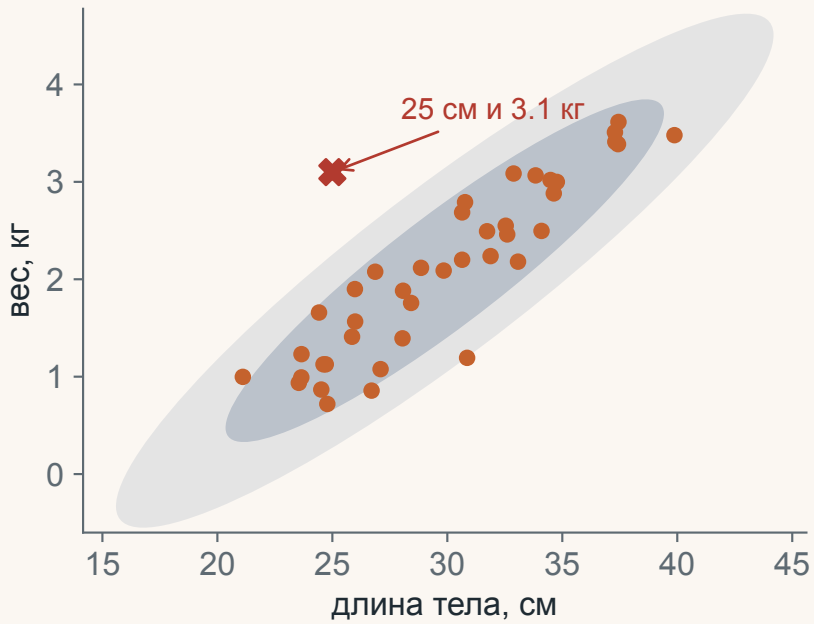
Три опечатки: добавим 35 и 38. Выбросы раздувают σ до 9.2, и у 33 кг $z = 2.8 < 3$ — проходит. Это **маскировка**: выбросы прячут друг друга.

IQR. Квартили зависят от середины данных, выбросы их почти не сдвигают: норма — от 1.95 до 6.35 кг. **Пойманы все три.**



Итог: три сигмы страдают от тех самых выбросов, которые ищут. Для первого фильтра надёжнее IQR.

По отдельности признаки в норме, а вместе — странно: помогает расстояние Махаланобиса



Котёнок: длина 25 см, вес 3.1 кг. Длина обычная ($z \approx -1.0$), вес тоже ($z \approx +1.2$). Но вместе — слишком тяжёлый для такой длины. По одному признаку его не поймать.

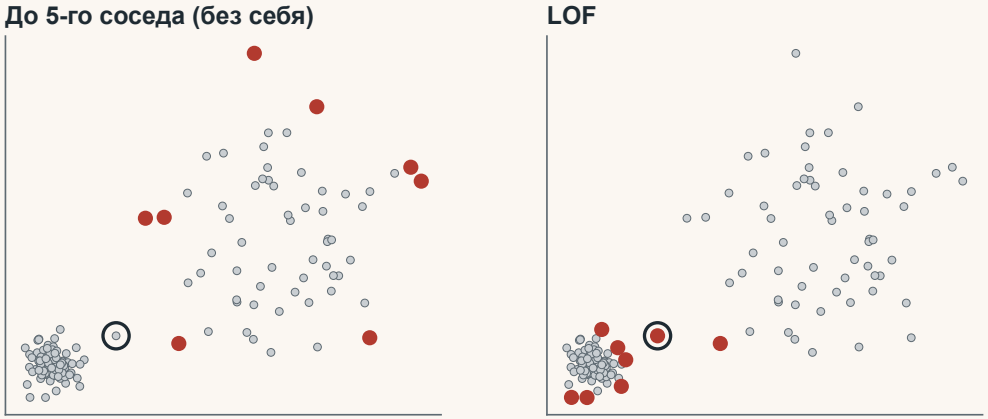
Расстояние Махаланобиса — расстояние до центра, измеренное в «эллипсах» облака, а не в кругах. Оно учитывает корреляцию признаков (Σ — матрица ковариаций):

$$d_M(x) = \sqrt{(x-\mu)^T \, \Sigma^{-1} (x-\mu)}$$

У нашего котёнка $d_M = 5.3$, у остальных — не больше 3.02.

В sklearn: `EllipticEnvelope`. Он оценивает μ и Σ устойчиво к выбросам, поэтому числа будут чуть другими. Подходит, когда данные — одно вытянутое облако.

Если облака разной плотности, расстояние до соседей ошибается, а LOF — нет



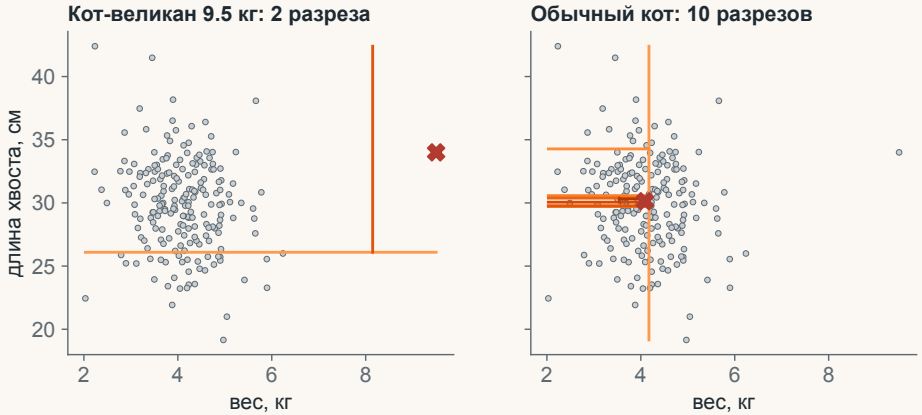
Красные — 8 самых подозрительных точек по каждому методу

Расстояние до 5-го соседа: далеко до соседей — подозрительно. В разреженном облаке далеко все — красным стало оно целиком, а точка в кружке лишь 12-я.

LOF (local outlier factor) сравнивает плотность вокруг точки с плотностью вокруг её соседей. LOF ≈ 1 — как у соседей, LOF ≫ 1 — ниже. У точки в кружке LOF = 3.9, первое место.

> **Итог:** LOF ищет точки, которые реже своих соседей, а не просто далёкие. Для новых объектов — `LocalOutlierFactor(novelty=True)`.

Isolation Forest: аномалию отрезают от остальных за пару случайных разрезов



Дерево режет данные **случайно**: случайный признак, случайный порог — пока точка не останется одна. Аномалию отрезают быстро, а обычную точку окружают соседи — разрезов нужно много. На рисунке — одно дерево.

В среднем по 2000 случайных деревьев: кот-великан — **3.2** разреза, обычный кот — **13.9**. Мера аномальности — средняя глубина по лесу: чем меньше, тем подозрительнее.

Плюсы: расстояний нет — масштабировать не нужно; быстро и на многих признаках. Версия упрощена: в sklearn дерево строится на подвыборке до 256 точек с ограниченной глубиной.

Уже знакомые методы тоже ищут аномалии: шум DBSCAN и ошибка восстановления PCA

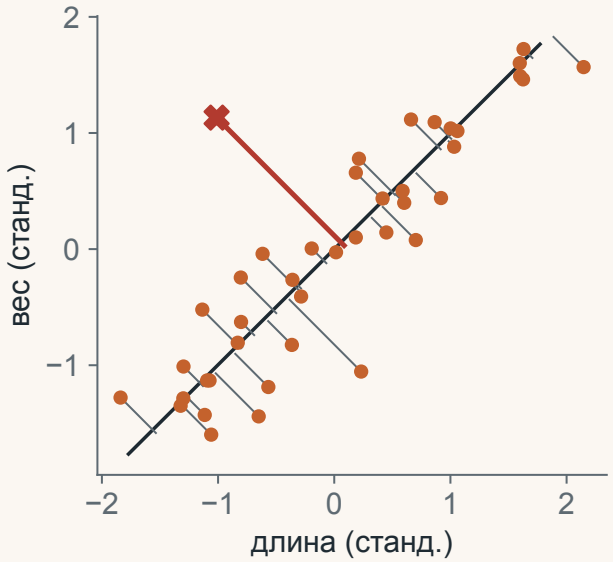
DBSCAN

Шумовые точки (метка −1) — готовые кандидаты в аномалии. Помните 5 одиноких заказов на окраине?

PCA: ошибка восстановления

Сжимаем объект до первых компонент и восстанавливаем обратно. Обычные объекты лежат вдоль главной оси и восстанавливаются почти точно. Непохожий на остальных — плохо.

Тот же котёнок 25 см и 3.1 кг: квадрат ошибки **2.42**, у остальных — не больше 0.77 (медиана 0.03).



Дальше: нелинейное обобщение этой идеи — автоэнкодер, нейросеть, которая сжимает и восстанавливает объект.

Accuracy здесь бесполезна: смотрим PR-AUC и precision@k, а порог выбираем по бюджету проверок

2000 обычных объектов и 20 аномалий. Модель «всё нормально» даёт **accuracy 99%** и не находит ни одной аномалии — как «всегда пёсик» в лекции 3.

Что смотреть

- **precision** — доля настоящих аномалий среди помеченных, **recall** — доля найденных среди всех аномалий
- **PR-AUC** — площадь под кривой precision–recall: 1 — идеально
- **precision@k** — доля настоящих аномалий среди k самых подозрительных

Isolation Forest: PR-AUC = **0.985** (случайный порядок дал бы около 0.01), precision@20 = **0.90**.

Порог. С `contamination="auto"` Isolation Forest пометил 228 объектов: нашёл все 20 аномалий, но 208 тревог — ложные.

Как выбрать порог

По маленькой размеченной выборке — или по бюджету: сколько тревог в день успеют проверить люди. Тогда берём столько самых подозрительных.

Какой метод выбрать: начинаем с IQR, хороший универсальный выбор — Isolation Forest

Метод	Идея	Когда брать	sklearn
Три сигмы, IQR	далеко от центра по одному признаку	быстрый первый фильтр	— (numpy, pandas)
Гауссов эллипс	далеко с учётом корреляций	данные — одно вытянутое облако	`EllipticEnvelope`
LOF	плотность ниже, чем у соседей	кластеры разной плотности	`LocalOutlierFactor`
Isolation Forest	легко отрезать случайными разрезами	много признаков, большие данные; хороший первый выбор	`IsolationForest`
DBSCAN	вне плотных областей	всё равно кластеризуем	`DBSCAN`
PCA	плохо восстанавливается	признаки сильно коррелированы	`PCA + inverse_transform`

```
from sklearn.ensemble import IsolationForest

iso = IsolationForest(n_estimators=300, contamination=0.01, random_state=0).fit(X_train)
iso.predict(X_new)          # −1 — аномалия, 1 — норма
iso.score_samples(X_new)    # чем меньше, тем подозрительнее
```

Что мы обсудили: три задачи обучения без учителя

1. **Без учителя** ищем структуру без y: группы, короткое описание, странные объекты.
2. **K-means:** шаг 1 — каждый объект к ближайшему центру, шаг 2 — центр в среднее группы. От локальных минимумов спасают k-means++ и `n_init`. K выбираем по локтю и силуэту. Признаки масштабируем.
3. **Иерархическая кластеризация и DBSCAN:** дендрограмма — без выбора K заранее; плотность — кластеры любой формы и шум.
4. **PCA:** оси с наибольшим разбросом = наименьшая ошибка восстановления; компоненты — собственные векторы матрицы ковариаций.
5. **SVD:** $X = U\Sigma V^T$ для любой матрицы; через него считают PCA, сжимают картинки и строят рекомендации.
6. **Аномалии:** три типа; IQR, Махаланобис, LOF, Isolation Forest, DBSCAN и PCA; качество — по PR-AUC и precision@k, не по accuracy.

Шпаргалка: какая задача — какой метод, и пять частых ошибок

Задача	Что попробовать
найти сегменты	K-means; при сложной форме или шуме — DBSCAN, HDBSCAN
понять, сколько групп	локоть, силуэт, дендрограмма
нарисовать многомерные данные	PCA; для картинки — t-SNE, UMAP
сжать признаки перед моделью	PCA (обучаем на train); для разреженных — TruncatedSVD
рекомендации	матричные разложения
найти странные объекты	Isolation Forest, LOF, EllipticEnvelope

Частые ошибки

- не отмасштабировали признаки
- обучили PCA на всех данных вместе с тестом
- сделали выводы о расстояниях по картинке t-SNE
- оценили поиск аномалий по accuracy
- поверили кластерам, не проверив их смысл

Посмотреть и потрогать: интерактивные демо и документация

- **Visualizing K-Means Clustering** — Naftali Harris: двигаем центры руками

  naftaliharris.com/blog/visualizing-k-means-clustering
- **Visualizing DBSCAN Clustering** — Naftali Harris

  naftaliharris.com/blog/visualizing-dbscan-clustering
- **Principal Component Analysis Explained Visually** — Setosa

  setosa.io/ev/principal-component-analysis
- **How to Use t-SNE Effectively** — Distill: почему картинкам t-SNE нельзя доверять буквально

  distill.pub/2016/misread-tsne
- **scikit-learn User Guide:** Clustering; Novelty and Outlier Detection

  scikit-learn.org/stable/modules/clustering.html · scikit-learn.org/stable/modules/outlier_detection.html

ДЛЯ ЛЮБОПЫТНЫХ

Приложение

Выводы и формулы, которые не поместились в лекцию.

Приложение. Вывод PCA: условие максимума даёт $Cw = \lambda w$

Ищем направление с наибольшим разбросом проекций $w^T C w$ при условии $\|w\| = 1$. Функция Лагранжа:

$$L(w, \lambda) = w^T C w - \lambda\,(w^T w - 1), \qquad \frac{\partial L}{\partial w} = 2Cw - 2\lambda w = 0 \;\Rightarrow\; Cw = \lambda w$$

- Значит, оптимальное w — собственный вектор C.
- Разброс вдоль него: $w^T C w = \lambda\, w^T w = \lambda$. Берём наибольшее λ_1.
- Вторая компонента: то же плюс условие $w \perp w_1$. Получаем собственный вектор со следующим по величине λ_2, и так далее.
- C симметрична и неотрицательно определена ($w^T C w \geq 0$ при любом w: разброс проекций не бывает отрицательным), поэтому все $\lambda_j \geq 0$, а собственные векторы можно выбрать ортонормированными.
- Доля дисперсии k первых компонент: $(\lambda_1 + \cdots + \lambda_k)/(\lambda_1 + \cdots + \lambda_d)$.

Приложение. Связь SVD и PCA: сжатие PCA — это усечённое SVD

Пусть X центрирована, $X = U\Sigma V^T$, $U^TU = V^TV = I$, $C = \frac{1}{\ell}X^TX$.

$$C = \tfrac{1}{\ell}\, V\Sigma U^T U\Sigma V^T = V\,\tfrac{\Sigma^2}{\ell}\,V^T \;\Rightarrow\; Cv_j = \tfrac{\sigma_j^2}{\ell}\,v_j$$

- Столбцы V — собственные векторы C, то есть главные компоненты; $\lambda_j = \sigma_j^2/\ell$.
- Координаты объектов в новых осях: $Z = XV = U\Sigma V^TV = U\Sigma$.
- Восстановление по k компонентам: $Z_kV_k^T = U_k\Sigma_kV_k^T = X_k$. **Сжатие PCA — это усечённое SVD.**
- Ошибка восстановления: $\|X - X_k\|_F^2 = \sigma_{k+1}^2 + \cdots + \sigma_r^2$ — сумма квадратов выброшенных сингулярных чисел ($\|A\|_F^2$ — сумма квадратов всех элементов матрицы).

Приложение. Смесь гауссиан — «мягкий» K-means с вероятностями

Каждый кластер — нормальное распределение со своим центром μ_k, матрицей ковариаций Σ_k и весом π_k. Кластер может быть вытянутым эллипсом, а объект принадлежит кластерам **с вероятностями**.

EM-алгоритм — те же два шага, что у K-means, только мягкие:

- **E-шаг** (мягкий шаг 1): для каждого объекта считаем вероятности принадлежности кластерам — вместо «к ближайшему центру»
- **M-шаг** (мягкий шаг 2): пересчитываем μ_k, Σ_k, π_k, взвешивая объекты этими вероятностями, — вместо «центр в среднее группы»

Связь с K-means

Если все кластеры — одинаковые круги и дисперсия стремится к нулю, вероятности становятся 0 или 1, и EM превращается в K-means.

В sklearn: `GaussianMixture(n_components=K)`; `predict_proba` — вероятности кластеров. Число кластеров можно выбирать по BIC.

Приложение. Формулы Isolation Forest и LOF

Isolation Forest

$$s(x) = 2^{-\,\mathbb{E}\,h(x)\,/\,c(n)}$$

- $h(x)$ — глубина, на которой точка изолирована; n — размер подвыборки на дерево (в sklearn по умолчанию до 256)
- $c(n) = 2H(n-1) - 2(n-1)/n$ — средняя глубина в случайном дереве, $H(i) \approx \ln i + 0.5772$
- $s \to 1$ — аномалия; s заметно меньше 0.5 — норма
- sklearn: `score_samples` возвращает $-s(x)$

LOF

$$\mathrm{lrd}(x) = \left(\frac{1}{k} \sum_{o \in N_k(x)} \mathrm{rd}_k(x, o) \right)^{-1}$$

$$\mathrm{LOF}(x) = \frac{1}{k} \sum_{o \in N_k(x)} \frac{\mathrm{lrd}(o)}{\mathrm{lrd}(x)}$$

- $N_k(x)$ — k ближайших соседей
- $\mathrm{rd}_k(x, o) = \max(r_k(o),\ \rho(x, o))$, где $r_k(o)$ — расстояние от объекта o до его k-го соседа; rd_k — сглаженное расстояние
- lrd — локальная плотность; LOF — во сколько раз плотность у соседей выше, чем у самой точки